

Chapter 6

apply the knowledge of data structures to other application domains like data compression

Data structures serve as foundational building blocks across computer systems. While basic implementations demonstrate individual operations such as insertion, deletion, and traversal, practical software applications demand composite configurations. Real-world engineering domains, including data compression, high-performance caching, database search engines, and network routing, rely on combining elementary structures into specialized software architectures.

Data compression algorithms reduce storage overhead and network bandwidth requirements by reorganizing data using specialized tree structures and priority queues. Memory management subsystems maintain real-time execution speeds through composite cache structures that pair key lookup tables with ordering lists. Search engines and network routers utilize specialized search trees and graph representations to evaluate millions of queries and routing decisions per second.

This chapter applies foundational data structures to solve algorithmic problems across system domains. By examining these implementations in context, this chapter demonstrates the practical application of asymptotic optimization, structural trade-offs, and object design principles introduced throughout this course.

6.1 Data Compression via Optimal Prefix Trees

Data compression is the process of encoding information using fewer bits than the original representation. Lossless data compression guarantees that the original data can be perfectly reconstructed from the compressed representation. A fundamental approach to lossless compression relies on variable-length character encoding, where frequently occurring characters are assigned shorter bit sequences, while rarer characters receive longer bit sequences.

To ensure unambiguous decoding, variable-length codes must satisfy the prefix property.

A **prefix code** is a coding system in which no valid code word is a prefix of any other valid code word. This property allows a continuous stream of compressed bits to be parsed sequentially without delimiter tokens.

6.1.1 Huffman Coding Algorithm

Huffman coding is an algorithmic method for constructing optimal prefix codes. The algorithm uses a min-heap priority queue to build a full binary tree bottom-up, based on the relative frequencies of input symbols.

The tree construction procedure follows a deterministic sequence:

1. Compute the occurrence frequency of each unique symbol in the input source.
2. Create a leaf node for each symbol and insert all leaf nodes into a min-heap priority queue ordered by frequency.
3. While the priority queue contains more than one node:
 - (a) Extract the two nodes with the lowest frequencies (q_1 and q_2).
 - (b) Create a new internal node with a frequency equal to the sum of q_1 and q_2 .
 - (c) Set q_1 as the left child and q_2 as the right child of the internal node.
 - (d) Insert the new internal node back into the min-heap.
4. The remaining single node in the priority queue represents the root of the complete Huffman tree.

Traversing the resulting binary tree assigns binary digits to edges: left branches represent bit 0 and right branches represent bit 1. The binary string accumulated from the root to a leaf node forms the unique variable-length prefix code for that symbol.

6.1.2 Worked Example: Encoding "ABRACADABRA"

Consider compressing the 11-character string "ABRACADABRA". Standard fixed-length ASCII encoding allocates 8 bits per character, requiring $11 \times 8 = 88$ bits.

First, tally character frequencies:

Table 6.1: Symbol Frequency Table for "ABRACADABRA"

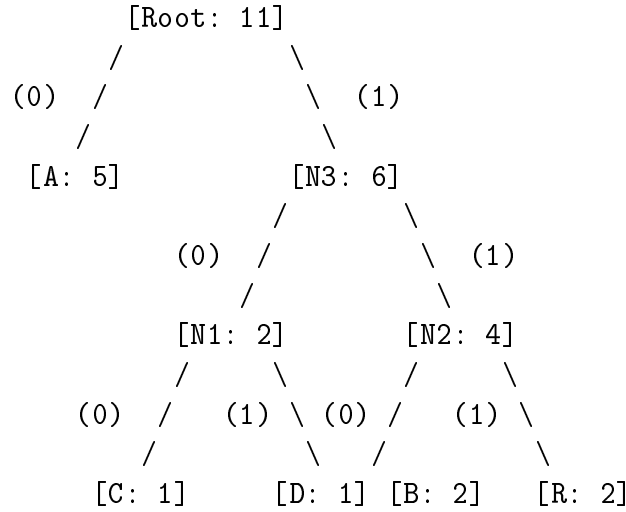
Symbol	Frequency
A	5
B	2
R	2
C	1
D	1

Next, construct the Huffman tree using a min-heap:

1. Initial min-heap contents (ordered by frequency): $[C(1), D(1), B(2), R(2), A(5)]$.

2. Extract $C(1)$ and $D(1)$. Create internal node N_1 with frequency $1 + 1 = 2$. Insert $N_1(2)$. Min-heap: $[B(2), R(2), N_1(2), A(5)]$.
3. Extract $B(2)$ and $R(2)$. Create internal node N_2 with frequency $2 + 2 = 4$. Insert $N_2(4)$. Min-heap: $[N_1(2), N_2(4), A(5)]$.
4. Extract $N_1(2)$ and $N_2(4)$. Create internal node N_3 with frequency $2 + 4 = 6$. Insert $N_3(6)$. Min-heap: $[A(5), N_3(6)]$.
5. Extract $A(5)$ and $N_3(6)$. Create root node R_{root} with frequency $5 + 6 = 11$.

The finalized Huffman binary tree layout is shown below:



Navigating from the root to each leaf yields the code table:

Symbol	Frequency	Huffman Code	Total Bits
A	5	0	5
B	2	110	6
R	2	111	6
C	1	100	3
D	1	101	3

The encoded string representation becomes:

0 (A) 110 (B) 111 (R) 0 (A) 100 (C) 0 (A) 101 (D) 0 (A) 110 (B) 111 (R) 0 (A)

The total storage consumed is $5 + 6 + 6 + 3 + 3 = 23$ bits. Compared to the uncompressed 88 bits, Huffman coding achieves a 73.86% space reduction for this message.

6.2 Composite Data Structures for High-Performance Caching

In-memory caching improves subsystem throughput by storing frequently queried data in fast RAM. Because cache memory capacity is limited, a cache must execute eviction

strategies when full. The **least recently used cache** (LRU cache) evicts the item that has not been accessed for the longest duration.

An LRU cache requires two operations to execute in $O(1)$ worst-case time complexity:

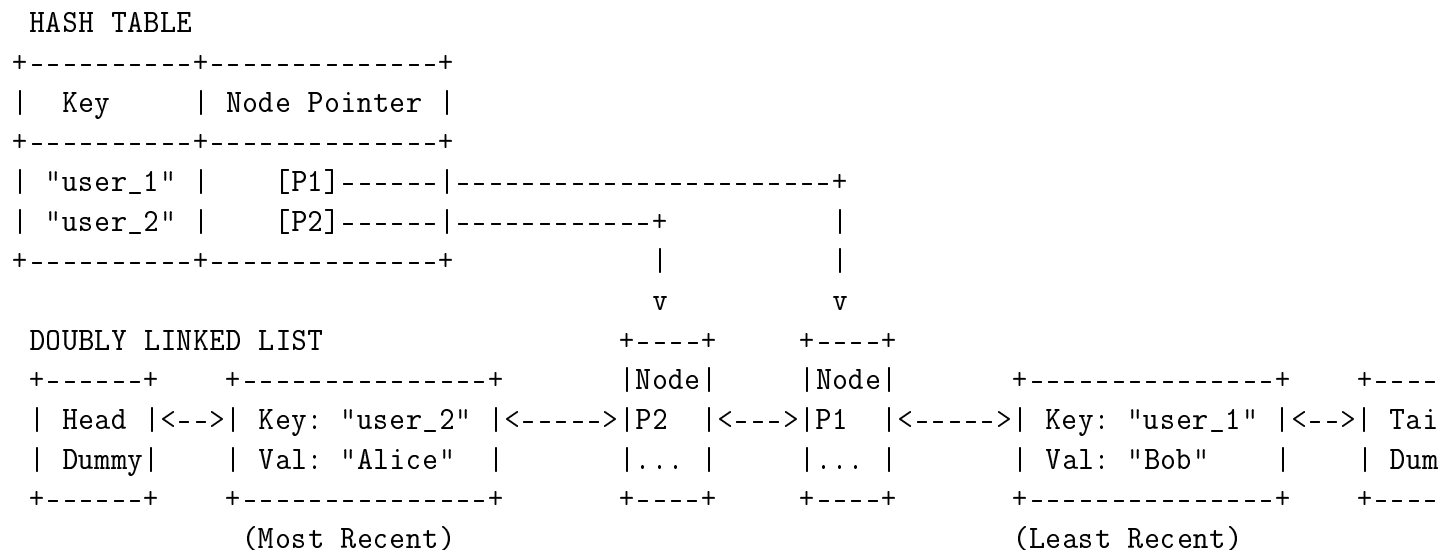
- `get(key)`: Retrieve the item value associated with the key and mark it as most recently used.
- `put(key, value)`: Insert or update a key-value pair. If capacity is exceeded, evict the least recently used item.

A single foundational data structure cannot satisfy both operational constraints simultaneously. Searching a doubly linked list requires $O(n)$ time, while a standard hash table cannot track item access recency in $O(1)$ time.

The optimal architectural solution couples a hash table with a doubly linked list.

6.2.1 LRU Cache Structural Integration

The doubly linked list maintains the items in sequence of access recency. The head of the list stores the most recently accessed item, while the tail stores the least recently accessed item. The hash table maps search keys directly to list node pointers, bypassing linear list traversal.



6.2.2 Operational Execution Trace

The composite LRU cache executes its core methods through joint state manipulation:

Get Operation

1. Query the hash table with the key. If the key is absent, return a cache miss status.
2. If the key is present, extract the direct memory pointer to the list node.
3. Detach the node from its current position in the doubly linked list by modifying its neighbors' pointers in $O(1)$ time.

4. Splice the node directly behind the list head node (marking it most recently used).
5. Return the stored value.

Put Operation

1. Query the hash table with the key. If the key exists, update its value field and promote the node to the list head.
2. If the key does not exist:
 - (a) If the cache count equals capacity, retrieve the node preceding the tail dummy node (the least recently used node).
 - (b) Delete the key of the tail node from the hash table.
 - (c) Remove the tail node from the doubly linked list in $O(1)$ time.
 - (d) Allocate a new node containing the key and value, splice it behind the head node, and add its pointer reference to the hash table.

6.3 Database Indexing and Search Engine Retrieval

Information retrieval and database systems process non-numeric data queries, such as text searches and prefix matching. Generic binary search trees or arrays are inefficient for large-scale string processing due to string comparison overheads. Specialized indexing structures address these operational constraints.

6.3.1 Inverted Indexing

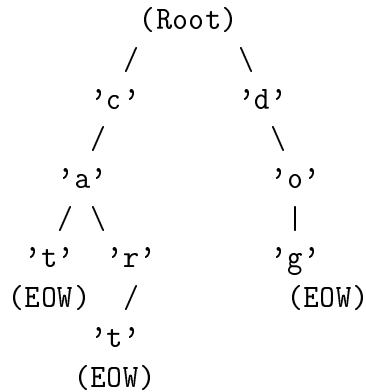
An **inverted index** is a dictionary structure optimized for text retrieval across large document collections. Instead of mapping documents to contained words, an inverted index maps distinct terms to a list of document identification tags where those terms appear. This structure is known as a postings list.

TERM DICTIONARY (Hash Table / Tree)	POSTINGS LISTS (Singly Linked Lists)
+-----+	+-----+ +-----+ +-----+
"algorithm" ----->	12 ---> 45 ---> 89
+-----+	+-----+ +-----+ +-----+
"binary" ----->	3 ---> 45
+-----+	+-----+ +-----+
"complexity" ----->	12 ---> 99
+-----+	+-----+ +-----+

Constructing an inverted index uses a hash table or search tree for the primary term dictionary, while storing postings as dynamic arrays or singly linked lists. To execute a boolean search query such as "algorithm AND binary", the engine retrieves the posting lists for both terms and performs a linear list intersection algorithm in time proportional to the length of the lists.

6.3.2 Prefix Search via Tries

A **trie** (also termed a prefix tree) is a tree-based search data structure where nodes represent character keys rather than entire data items. The root node represents an empty key, and child nodes represent single-character extensions.



In the trie above, the stored words are "cat", "cart", and "dog". Nodes labeled (EOW) represent the end of a valid word.

Tries enable optimal string searching and auto-complete functionality. Searching for a string of length k takes $O(k)$ time complexity, independent of the total number of keys stored in the dataset.

6.4 Data Structure Selection in Applied Domains

Selecting data structures requires evaluating operations, time complexities, and memory requirements against application objectives. Table 6.3 compares the domain-specific data structures discussed in this chapter.

6.5 Graph Structures in Network Routing

Graph structures model connected networks such as the internet, road systems, and social graphs. Modern routing protocol engines rely on efficient graph representations coupled with priority queues to evaluate optimal signal pathways.

Dijkstra's shortest path algorithm calculates the minimum path distance from a single source vertex to all other vertices in a weighted graph.

The graph representation dictates algorithmic efficiency:

- **Adjacency Matrix:** An $n \times n$ matrix providing $O(1)$ edge weight lookups, but requiring $O(V^2)$ total space complexity.
- **Adjacency List:** Array of linked lists storing only existing edges, consuming $O(V + E)$ space complexity, where V is vertices and E is edges.

SAMPLE GRAPH

ADJACENCY MATRIX

ADJACENCY LIST

Table 6.3: Comparison of Domain-Specific Data Structures

Structure	Core Operations	Time Complexity	Primary Domain
Huffman Tree	Bit insertion, Prefix code parsing	$O(n \log k)$ build, $O(k)$ decode per character	Lossless text and media compression
LRU Cache	Key lookups, Node promotion and eviction	$O(1)$ for get and put	In-memory performance caching
Inverted Index	Dictionary lookup, List intersection	$O(k)$ key search, $O(L_1 + L_2)$ intersection	Full-text search engine indexing
Trie	String search, Prefix matching	$O(k)$ key lookup ($k =$ string length)	Auto-complete, IP packet routing

(A) --- (B)

| /

| /

(C) --- (D)

A B C D

A 0 1 1 0

B 1 0 1 1

C 1 1 0 0

D 0 1 0 0

[A] -> B -> C

[B] -> A -> C -> D

[C] -> A -> B

[D] -> B

When Dijkstra's algorithm uses a flat array to scan unvisited vertices, the minimum distance selection takes $O(V)$ time per vertex, producing an overall runtime complexity of $O(V^2)$.

By pairing an adjacency list graph representation with a min-heap priority queue, finding the minimum distance vertex executes in $O(\log V)$ time. This optimization reduces the overall algorithm execution time complexity to $O((V + E) \log V)$, enabling real-time routing over massive networks.

Tutorial Questions

1. Explain the prefix property in variable-length code systems. Describe the functional problems that arise during decoding if a set of codes violates this property.
2. Construct a Huffman code tree for a document containing the following character counts: E: 12, T: 9, A: 8, I: 4, O: 2. List the binary code generated for each character and compute the total bits required to encode the source document.
3. An LRU cache with a total capacity of 3 items processes the following sequence of execution calls:

```
put("A", 1), put("B", 2), put("C", 3), get("A"), put("D", 4)
```

Draw the contents of both the hash table and the doubly linked list after each operation completes. Identify which key is evicted upon inserting ("D", 4).

4. Compare the storage efficiency and time performance of a hash table versus a trie for storing a dictionary of 500,000 text strings. Highlight conditions under which a trie is preferred.
5. Explain how an inverted index executes multi-word intersection queries (such as "data AND structure"). State the worst-case time complexity of intersecting two posting lists of lengths M and N .
6. Analyze the algorithmic impact of data structure selection in Dijkstra's shortest path algorithm. Contrast the theoretical performance of an implementation using a linear array versus one using a min-heap priority queue.
7. A network router needs to match incoming IP addresses to a table of 100,000 network prefixes in real time. Choose an appropriate data structure for this application domain, justify your selection over alternative structures, and analyze its search time complexity.

Conclusion

Data structures are non-isolated theoretical models; they serve as structural primitives that power software engineering solutions across modern applications. As demonstrated throughout this chapter, complex domain applications, including lossless compression engines, operating system memory caches, full-text search indexes, and network packet routers, require combining foundational data structures to meet real-world computational constraints. Combining hashing, tree models, linked sequences, and priority queues allows systems to optimize execution time and space utilization simultaneously, translating theoretical complexity bounds into scalable software architectures. Having examined how composite data structures optimize resource efficiency in production systems, the next step is to explore how algorithmic design paradigms—such as greedy strategies, divide-and-conquer, and dynamic programming—leverage these underlying structures to solve complex computational problems.